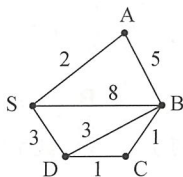


提高组 CSP-S 2024 初赛模拟卷 5

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 以下哪个命令会开启编译器几乎所有常用的警告？（ ）
A. `g++ -g -lm main.cpp -o main`
B. `g++ -g -Wall main.cpp -o main`
C. `g++ -g -O2 main.cpp -o main`
D. `g++ -g -std=c++11 main.cpp -o main`
2. 以下关于 NOI Linux 的文件操作描述错误的是（ ）。
A. `mkdir` 是新建一个文件
B. `pwd` 显示当前工作文件路径
C. `ls` 显示文件及文件夹
D. `more` 在终端中分页显示普通文本类型文件
3. 以下哪个不属于 STL 中算法模板库的操作函数？（ ）
A. `reverse` B. `sort` C. `push_back` D. `lower_bound`
4. 以下哪个说法是不正确的？（ ）
A. `tuple` 中的每个值都必须是相同的类型
B. `pair` 定义在头文件 `utility` 中，用于将两个值组合在一起存储
C. `multiset` 是维护有序可重集合的容器
D. `set` 是维护有序不可重集合的容器
5. 对下图使用 Floyd 算法计算 S 点到其余各点的最短路径长度时，到 B 点的距离 $d[B]$ 初始时赋为 8，在算法的执行过程中不可能出现的值是（ ）。
A. 4 B. 5 C. 6 D. 7



6. 平面上有 5 个点 $A(5, 3)$, $B(3, 5)$, $C(2, 1)$, $D(3, 3)$, $E(5, 1)$ 。以这 5 个点作为完全图 G 的顶点, 每两点之间的直线距离是图 G 中对应边的权值。图 G 的最小生成树中的所有边的权值和为 ()。

A. 8 B. $7+\sqrt{5}$ C. 9 D. $6+\sqrt{5}$

7. 单源最短路的 Dijkstra 算法写的 C++ 程序的运行时间复杂度是 ()。

A. $O(n^2 \log n)$ B. $O(n^3)$ C. $O(n^2)$ D. $O(n \log n)$

8. 对于一棵二叉树, 独立集是指两两互不相邻的结点构成的集合。比如, 图 1 有 5 个不同的独立集 (1 个双点集合、3 个单点集合、1 个空集), 图 2 有 14 个不同的独立集。那么, 图 3 有 () 个不同的独立集。

A. 1936 B. 3600 C. 5536 D. 6300



图 1

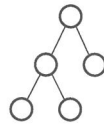


图 2

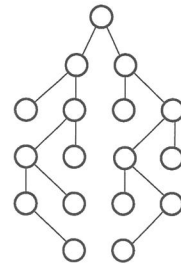


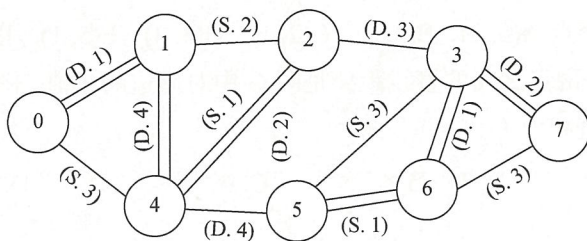
图 3

9. 以下哪个说法是正确的? ()

A. 要判断图是否连通, 只能用广度优先搜索算法来判别
 B. 高斯是图论的创始人之一, 他解决了哥尼斯堡七桥问题
 C. 图灵的主要贡献是用图灵机破解了德军的 Enigma 密码
 D. 如果待查找的元素确定, 只要哈希表的大小不小于查找元素的个数, 就一定存在不会产生冲突的哈希函数

10. 下图中的 8 个结点通过 13 条无向边相连, 每条无向边上都有两项信息: 前面的字母表示这条边的安全性 (字母 S 表示这条边是 Safe 边, 字母 D 表示这条边是 Danger 边), 后面的数字表示这条边的长度。从中找出 7 条边能连通这 8 个结点, 且这 7 条边中恰好有 2 条 Safe 边 (S 边), 记这 7 条边长度之和为 path, 则 path 的最小值为 ()。

A. 10 B. 11 C. 12 D. 13



11. 以下哪个不是 C++ 语言中面向对象模式的主要性质? ()
- A. 封装性 B. 继承性 C. 单调性 D. 多态性
12. 从 1 到 2021 的所有奇数中, 至少要选出 () 个数, 才能确保其中必定存在两个数, 它们的和是 2022?
- A. 505 B. 506 C. 507 D. 508
13. 如果一棵二叉树有 2024 个结点, 树中的结点按层次和顺序依次编号 (每层中从左到右编号), 其中根结点为 1, 那么编号为 1025 的父结点为第 () 层第 () 个结点。
- A. 10 1 B. 10 2 C. 9 256 D. 11 2
14. 4 个学生和 2 个老师围绕圆桌入座, 2 个老师之间至少有 1 个学生, 有多少种就座方法? ()
- A. 36 B. 72 C. 48 D. 84
15. 由数字 1, 2, 3, 4, 5, 6, 7 组成无重复数字的 7 位整数, 从中任取一个, 所取的数满足首位是 1, 且任意相邻两位数字之差的绝对值不大于 2, 则取到此类数的概率为 ()。
- A. 1/120 B. 1/240 C. 1/360 D. 1/480

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include <iostream>
02 #include <algorithm>
03 #include <string.h>
04
05 using namespace std;
```

```
06
07 const int N = 100000, M = 100000;
08 const long long mod = 1e8-3;
09
10 struct StMatch{
11     int a, b;
12 } matches[N+1];
13 bool operator < (const StMatch a, const StMatch b) {
14     return a.a < b.a;
15 }
16
17 struct StTreeArray {
18     int tree[M+1];
19     int lb(int x) { return x & (-x); }
20     void add(int x, int t) {
21         while (x <= M) {
22             tree[x] += t;
23             x += lb(x);
24         }
25     }
26     int sum(int x) {
27         int ans = 0;
28         while (x > 0) {
29             ans += tree[x];
30             x -= lb(x);
31         }
32         return ans;
33     }
34 };
35
36 StTreeArray t;
37 int n;
38 int a[N+2], b[N+2], pos[N+2];
39
40 int index(int a[], int x) {
41     int l = 1, r = n;
42     while (l < r) {
43         int m = (l + r) >> 1;
44         if (a[m] == x) return m;
45         if (a[m] < x) {
```

```
46         l = m + 1;
47     } else {
48         r = m - 1;
49     }
50 }
51 if (a[l] == x) return l;
52 if (a[r] == x) return r;
53 return 0;
54 }
55
56 int main() {
57     cin >> n;
58     for (int i = 1; i <= n; i++) {
59         cin >> matches[i].a;
60         a[i] = matches[i].a;
61     }
62     for (int i = 1; i <= n; i++) {
63         cin >> matches[i].b;
64         b[i] = matches[i].b;
65     }
66     sort(a+1, a+n+1);
67     sort(b+1, b+n+1);
68     for (int i = 1; i <= n; i++) {
69         matches[i].a = index(a, matches[i].a);
70         matches[i].b = index(b, matches[i].b);
71         pos[matches[i].b] = i;
72     }
73     for (int i = 1; i <= n; i++) {
74         matches[i].a = pos[matches[i].a];
75     }
76     long long ans = 0;
77     for (int i = n; i >= 1; i--) {
78         ans += t.sum(matches[i].a - 1);
79         ans %= mod;
80         t.add(matches[i].a, 1);
81     }
82     cout << ans << endl;
83     return 0;
84 }
```


■ 判断题

16. 第 19 行就是计算 lowbit, 即返回 x 表示成二进制后最右边的 1 在哪一位。 ()
17. 将第 13、14 行中的 “<” 同时或者其中之一改成 “>”, 不影响程序结果。 ()
18. 第 52 行可以去掉, 不影响程序结果。 ()
19. 第 69 行也可以去掉, 不影响程序结果。 ()

■ 选择题

20. 本程序的时间复杂度是 ()。
- A. $O(n^2 \log n)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$
21. 对于输入 "4 1 3 4 2 1 7 2 4", 本程序的输出为 ()。
- A. 1 B. 2 C. 3 D. 4

(2)

```
01 #include<cstdio>
02 using namespace std;
03 int gcd(int a,int b) {
04     return b==0?a:gcd(b,a%b);
05 }
06 int main() {
07     int T;
08     scanf("%d",&T);
09     while(T--) {
10         int a0,a1,b0,b1;
11         scanf("%d%d%d%d",&a0,&a1,&b0,&b1);
12         int p=a0/a1,q=b1/b0,ans=0;
13         for(int x=1;x*x<=b1;x++)
14             if(b1%x==0){
15                 if(x%a1==0&&gcd(x/a1,p)==1&&gcd(q,b1/x)==1) ans++;
16                 int y=b1/x;
17                 if(x==y) continue;
18                 if(y%a1==0&&gcd(y/a1,p)==1&&gcd(q,b1/y)==1) ans++;
19             }
20         printf("%d\n",ans);
21     }
22     return 0;
23 }
```

■ 判断题

22. 将第 17 行中的 `continue` 改为 `break`, 不影响程序结果。 ()
23. 如果输入是 "1 41 1 96 288", 则输出是 "5"。 ()
24. 第 13 行中的 `x=1` 可以换为 `x=a1`, 程序的运行速度一定会更快。 ()
25. 本题有时间复杂度更优的做法。 ()

■ 选择题

26. 本题实质上是求符合下面哪个条件的 x 的个数? ()
- A. $\gcd(a_0, x) = a_1$ 且 $\text{lcm}(x, b_1) = b_0$
- B. $\text{lcm}(a_1, x) = a_0$ 且 $\text{lcm}(x, b_0) = b_1$
- C. $\gcd(a_0, x) = a_1$ 且 $\gcd(x, b_1) = b_0$
- D. $\gcd(a_0, x) = a_1$ 且 $\text{lcm}(x, b_0) = b_1$
27. 如果输入是 "1 95 1 37 1776", 则输出是 ()。
- A. 1 B. 2 C. 3 D. 4

(3)

```
01 #include <iostream>
02 #include <string.h>
03 #include <math.h>
04 #include <algorithm>
05 using namespace std;
06
07 const int N = 10010, MONEY = 1010, L = 1010;
08
09 struct sLine{
10     int st, en, fun, cost;
11 };
12 int l, n, money;
13 sLine line[N];
14 int f[L][MONEY], use[L][MONEY], jump[L][MONEY];
15 void print(int l, int cost);
16 bool comp(sLine & la, sLine & lb) {
17     return (la.en < lb.en ||
18         (la.en == lb.en && la.st < lb.st));
19 }
20
```

```
21 int main() {
22     cin >> l >> n >> money;
23     for (int i = 0; i < n; i++) {
24         cin >> line[i].st >> line[i].en
25             >> line[i].fun >> line[i].cost;
26         line[i].en += line[i].st;
27     }
28     sort(line, line + n, comp);
29
30     memset(f, 128, sizeof(f));
31     f[0][0] = 0; jump[0][0] = -1; use[0][0] = -1;
32     for (int i = 0; i < n; i++) {
33         for (int c = money; c >= line[i].cost; c--) {
34             int st = line[i].st;
35             if (f[st][c - line[i].cost] + line[i].fun >
36                 f[line[i].en][c]) {
37                 jump[line[i].en][c] = st;
38                 use[line[i].en][c] = i;
39             }
40             f[line[i].en][c] = max(f[line[i].en][c],
41                 f[st][c - line[i].cost] + line[i].fun);
42         }
43     }
44     int ans = -1, sol = 0;
45     for (int c = 0; c <= money; c++) {
46         if (f[l][c] > ans) {
47             sol = c;
48         }
49         ans = max(ans, f[l][c]);
50     }
51     if (ans >= 0) {
52         cout << ans << endl;
53         cout << "sol = " << sol << endl;
54         print(l, sol);
55     }
56     else cout << -1 << endl;
57     return 0;
58 }
59 void print(int l, int cost) {
60     if (l <= 0) return ;
61     print(jump[l][cost], cost - line[use[l][cost]].cost);
```



```
62     cout << "line : " << use[l][cost]
63     << " st : " << line[use[l][cost]].st
64     << " en : " << line[use[l][cost]].en
65     << " fun : " << line[use[l][cost]].fun
66     << " cost : " << line[use[l][cost]].cost << endl;
67     return ;
68 }
```

■ 判断题

28. (2分) 第 16~19 行是按照线段的终点排序, 且终点越大的越靠前, 终点相同则看起点。 ()
29. (2分) 第 30 行中的 128 可以换成别的数。 ()
30. (2分) 第 33 行倒序进行动态规划, 是为了节约空间, 使得 f 数组可以少一维。 ()
31. (2分) 本题涉及线段的代价和收益, 其实也可以使用最小费用最大流来解决。 ()

■ 选择题

32. (4分) 若 N 、 M 、 L 分别为最大线段数、最大代价限制、最长区间长度, 则本程序的时间复杂度为 ()。
- A. $O(NL)$ B. $O(NML)$ C. $O(ML)$ D. $O(MN)$
33. (4分) 如果使用最小费用最大流来解决本题, 则和本程序相比, 网络流解法的时间复杂度 ()。
- A. 不能使用网络流来解决 B. 更大
- C. 更小 D. 无法确定

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 给定数 a 和 b , 对 a 可以进行 3 种操作: $\times 2$, $/2$, $+1$ 。问: 最少要进行多少次操作可以让 a 变为 b ? 比如, 输入数据为 31 13, 输出为 8, 最佳操作顺序为: $31 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 13$ 。

本题不能简单地进行深度搜索, 因为前两个操作正好相互抵消, 可以无限搜索下去。

本程序首先使 a 、 b 满足 $a < b < 2a$, 然后尝试如何从 a 变成 b 。

```
01 #include<bits/stdc++.h>
02 #define int long long
03 using namespace std;
04 signed main() {
05     int a,b;
06     scanf("%lld%lld",&a,&b);
07     int cnt = 0;
08     while(a > b) {
09         cnt++;
10         if(a % 2 == 1) ①;
11         else a /= 2;
12     }
13     while(b >= a * 2) {
14         cnt++;
15         if(b % 2 == 1) ②;
16         else b /= 2;
17     }
18     if(③) {
19         printf("%lld\n",cnt);
20         return 0;
21     }
22     int aa = a,bb = b;
23     int minn=0x3f3f3f3f, bs=0;
24     while(aa>=1 && bb>=1 && aa<=bb) {
25         minn=min(minn, cnt+bs+(bb-aa));
26         if(aa % 2 == 1) ④;
27         aa /= 2;
28         bs++;
29         if(bb % 2==1) ⑤;
30         bb /= 2;
31         bs++;
32     }
33     printf("%lld\n",minn);
34     return 0;
35 }
```

34. ①处应填 ()。

A. a++

B. a--

C. a*=2

D. a/=2

35. ②处应填 ()。

A. b++

B. b--

C. b*=2

D. b/=2

36. ③处应填 ()。

A. a = b

B. a == b-1

C. a = b-1

D. a == b

37. ④处应填 ()。

A. aa--,bs--

B. aa--,bs++

C. aa++,bs--

D. aa++,bs++

38. ⑤处应填 ()。

A. bb--,bs--

B. bb++,bs++

C. bb--,bs++

D. bb++,bs--

(2) 输入一个字符串 s , 求其删掉若干个字符之后, 最多能出现多少个 “jessie”, 并且, 如果给出删掉每个字符的代价, 则需求出在保证代价最小的情况下 “jessie” 最多出现多少次。

比如, 输入数据为:

jesssie

1 1 5 4 6 1 1

输出为:

1

4

(因为 3 个 s 的代价分别为 5、4、6, 所以删掉的是第二个 s)

又比如, 输入数据为:

jejesconsiete

6 5 2 3 6 5 7 9 8 1 4 5 1

输出为:

1

21

```
01 #include <cstdio>
02 #include <cstring>
03 #include <iomanip>
04 #include <iostream>
05 using namespace std;
06 const int maxn = 300005;
```

```
07 char s[maxn];
08 int c[maxn][8], t[maxn][8], p[8][maxn], d[maxn];
09 int dp[maxn][8], sum[maxn], ans[maxn];
10 string jes = "jessie";
11 int main() {
12     scanf("%s", s + 1);
13     int n = strlen(s + 1);
14     for (int i = 1; i <= n; i++) {
15         cin >> d[i];
16         sum[i] = sum[i - 1] + d[i];
17     }
18     ①;
19     for (int i = 1; i <= n; i++) {
20         memcpy(dp[i], dp[i - 1], sizeof(int) * 8);
21         if (s[i] == 'j') {
22             dp[i][1] = max(dp[i][1], dp[i][0]);
23         } else if (s[i] == 'e') {
24             dp[i][2] = max(dp[i][2], dp[i][1]);
25             dp[i][6] = max(dp[i][6], dp[i][5]);
26             ②;
27         } else if (s[i] == 's') {
28             dp[i][4] = max(dp[i][4], dp[i][3]);
29             dp[i][3] = max(dp[i][3], dp[i][2]);
30         } else if (s[i] == 'i') {
31             dp[i][5] = max(dp[i][5], dp[i][4]);
32         }
33     }
34     cout << dp[n][6] << endl;
35     memset(p, 0x7f, sizeof(p));
36     memset(t, 0x7f, sizeof(t));
37     memset(c, 0x7f, sizeof(c));
38     memset(ans, ③, sizeof(ans));
39     t[0][0] = sum[n];
40     p[0][1] = sum[n];
41     ans[0] = 0;
42     for (int i = 1; i <= n; i++) {
43         p[0][1] = sum[n] - sum[i - 1];
44         for (int x = 6; x >= 1; x--) {
45             if (jes[x] == s[i]) {
46                 if (x == 1)
47                     c[i][x] = ans[dp[i][x] - 1];
48                 else
```

39. ①处应填 ()。

40. ②处应填 ()。

41. ③处应填 ()。

42. ④处应填()。

43. ⑤处应填()。

- A. `min(ans[dp[i][x]], t[i][x])`
 B. `min(ans[dp[i][x]], p[i][x])`
 C. `min(ans[dp[i][x]], d[i][x])`
 D. `min(ans[dp[i][x]], c[i][x])`